University of Sheffield

School of Electrical and Electronic Engineering

FPGA-Based CNN Inference Using An Adapted ResNet18 for MNIST Dataset(s)

May 2025

Supervisor:

A dissertation submitted in partial fulfilment of the requirements for the degree of Bachelors of Engineering (BEng) in Electrical and Eletronic Engineering

Abstract

A quantised ResNet18 convolutional neural network (CNN) accelerator is implemented for real-time, low-power digit classification on the Zybo Z7 FPGA platform. The design incorporates quantisation-aware training, network depth reduction, and channel compression to balance accuracy with hardware efficiency. Each convolutional layer is mapped to a dedicated streaming processing element in a pipelined architecture synthesized using High-Level Synthesis (HLS), with all intermediate data stored in on-chip memory. The system achieves 98.7% accuracy on the MNIST dataset at 56.73 frames per second and 39.67 FPS/W power efficiency. Limitations arising from the HLS toolchain and dataset-model mismatch are discussed, with proposed extensions including multi-clock domain execution and bitsliced arithmetic. The modular inference-oriented design supports integration into broader training workflows and scalable deployment on higher-resource platforms.

Key Points:

- FPGA implementation of CNN inference using ResNet18.
- Optimised for energy-efficient, real-time digit recognition tasks.
- Evaluated on MNIST against comparable studies.
- Key metrics: accuracy, latency, resource utilisation, power efficiency.
- Results show significant improvements in power efficiency and latency.

Individual Contributions

This dissertation is the result of my individual work. All content, analysis, and implementations presented herein are my own unless otherwise stated and appropriately referenced.

Acknowledgments

I would like to express my gratitude to Dr. Tiantai Deng and PhD Candidate Kaiyuan Yang for their guidance and support throughout this project. I also thank the Department of Electrical and Electronic Engineering at the University of Sheffield for providing the necessary resources and infrastructure to conduct this research.

Contents

1 Introduction **5**

2 Theory and Literature Review **9**

2.1 Introduction to Deep Neural Networks (DNNs) 9

2.2 CNN Architectures and Dataset Justification 9

2.3 Platform Suitability and Hardware Selection Rationale 10

2.4 Hardware-Optimized CNN Design and HLS 11

2.5 Review of Notable Works 12

2.6 Research Gaps and Limitations 12

3 Methodology **14**

3.1 CNN Model Development and Training 14

3.2 FPGA Platform and Tools 16

3.3 FPGA IP Core Design 17

3.3.1 Convolutional Layer (CBN(R)) 18

3.3.2 Pooling Layer 20

3.3.3 Final Connected Layer 21

3.3.4 Softmax/Argmax and Output Logic 21

3.4 System Integration 22

4 Results and Evaluation **24**

4.1 Discussion 27

5 Conclusion and Future Work **29**

5.1 Economic, Legal, Social, Ethical and Environmental Context 31

A Appendix: Additional Materials **32**

A.1 Algorithms and Pseudocode 32

A.2 Figures 35

List of Figures

1.1 ResNet18 latency and throughput on FPGA vs GPU across batch sizes. Adopted from [1]. 6
1.2 (a) MNIST handwritten digits 7
1.3 (b) UAE license plate (digit-focused) 7
1.4 Illustration of the MNIST dataset and a suitable application; a vehicle license plate. . 7
1.5 Thesis structure and implementation flow. 8

3.1 ResNet18 model adapted for MNIST. 14
3.2 Evaluation of accuracy versus channel simplification factor. A 16× simplification factor was selected based on this analysis. 15
3.3 Comparison of quantisation-Aware Training methods: tensor-wise versus channel-wise quantisation. Tensor-wise was selected for its superior memory efficiency. . . . 15
3.4 Hardware-suitable ResNet18 architecture with fused layers for FPGA execution. . . . 16
3.5 Flowchart depicting layer-wise IP core development flow. 17
3.6 Line and window buffering in convolution. 19
3.7 System-level architecture showing pipelined convolutional blocks and AXI interface connections. 23

4.1 GPU PPA metrics for ResNet18 on MNIST across batch sizes. 24

A.1 Dual-clock architecture showing memory and compute stages operating under separate clocks. High-frequency input feeds line and window buffers, while convolution executes in a lower-frequency domain. CDC logic ensures safe data transfer. 35
A.2 Bitsliced multiplier and BRAM coefficient packaging. Coefficients are pre-packed into partitioned BRAMs aligned to the unrolling factor (UF), enabling parallel access for sliced DSP inputs. The DSP computes a fused result, which is then unpacked using post-processing logic (masking and shifting) to extract valid sub-results. 36

List of Tables

3.1 Zybo Z7-20 Resource Metrics 17
3.2 Comparison of output-first vs. partially unrolled convolution implementations. 20

4.1 FPGA Synthesis Results Summary 25
4.2 Platform Comparison: FPGA vs. GPU for ResNet18 on MNIST 25
4.3 Comparison of FPGA-based CNN Inference on MNIST 28

5.1 Comparison of model simplification degrees and their impact on accuracy. 30

Chapter 1

Introduction

CNNs have revolutionised machine perception tasks across domains such as autonomous vehicles, surveillance, and medical imaging, owing to their ability to learn hierarchical features from structured visual data. A CNN is a type of deep learning model designed to process grid-like data, such as images, through layers of convolutional filters that detect local patterns, ranging from edges to complex textures. As the data progresses through successive layers, these local features are combined into more abstract representations, enabling tasks such as image classification, object detection, and segmentation with minimal manual feature engineering.

Among CNN architectures, ResNet18 stands out for its balance between computational efficiency and representational capacity. Introduced as part of the Residual Network (ResNet) family, ResNet18 incorporates skip connections that allow information to bypass certain layers, enabling deeper networks without the vanishing gradient problem. This architecture delivers strong accuracy while remaining lightweight enough for deployment on power- and resource-constrained platforms. Its popularity in both academic research and industrial applications highlights its suitability for real-time tasks, including digit recognition, visual search indexing, and embedded vision systems [1].

However, as AI workloads continue to scale across industries, concerns about their energy footprint are becoming increasingly urgent. Data centers are projected to account for up to 8% of global electricity consumption by 2030, with AI inference workloads contributing a significant and growing share. This underscores the importance of developing hardware-efficient inference methods that maintain high performance while minimizing energy usage, especially for edge devices that operate under strict power and latency constraints.

Field-Programmable Gate Arrays (FPGAs) have emerged as a compelling solution in this context. Unlike general-purpose processors or GPUs, FPGAs offer a reconfigurable hardware fabric that can be tailored to specific workloads, enabling highly parallelised and energy-efficient data processing. Their advantages are evident in real-world deployments, such as Alibaba's large-scale FPGA inference infrastructure, where ResNet18 running on FPGAs achieved significantly lower latency and higher throughput at low batch sizes compared to GPUs (Figure 1.1) [1]. These characteristics make FPGAs especially suitable for real-time streamed AI applications where responsiveness and efficiency are critical.

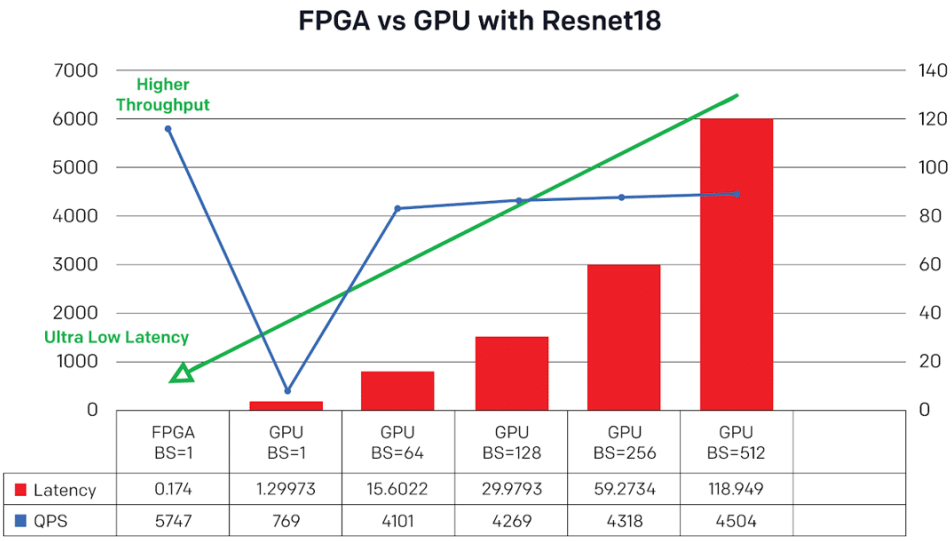

Figure 1.1: ResNet18 latency and throughput on FPGA vs GPU across batch sizes. Adopted from [1].

This thesis explores the FPGA-based deployment of a quantised ResNet18 architecture, adapted for the MNIST handwritten digit classification task. MNIST (Modified National Institute of Standards and Technology database) is a widely used benchmark dataset in machine learning research, containing 70,000 grayscale images of handwritten digits from 0 to 9, each with a resolution of 28×28 pixels. It is divided into 60,000 training and 10,000 test samples. Due to its simplicity, structure, and historical importance, MNIST is frequently used in early-stage research and hardware prototyping. Its small size and well-understood behavior enable rapid development and consistent benchmarking, while still reflecting challenges relevant to real-world classification tasks.

By focusing on MNIST, this thesis demonstrates an efficient CNN hardware accelerator that is both realistic in its deployment potential and comparable to existing research. The project leverages fixed-point quantisation to reduce arithmetic complexity, and implements the network using high-level synthesis (HLS) tools that generate FPGA-ready logic. Particular attention is given to resource-aware architectural design, optimizing key components such as convolutional layers, pooling, and activation functions for power-efficient inference on edge devices.

The primary objectives of this work are to (1) investigate architectural strategies for optimizing CNN inference on FPGAs, (2) evaluate trade-offs between accuracy, latency, and hardware utilisation, and (3) provide a reproducible design methodology suitable for edge AI deployment. Through this study, the thesis contributes toward the development of scalable, energy-conscious AI systems for the next generation of intelligent embedded platforms.

To guide the implementation and evaluation of this system, four project goals were defined in alignment with academic benchmarks established over the past five years:

1. Develop and train a CNN model (ResNet18) for MNIST digit classification, targeting a software accuracy of 99% or higher.

2. Deploy the model to an FPGA with an inference accuracy of at least 95%.

3. Achieve real-time classification performance, defined as a minimum of 30 frames per second (fps).

4. Reduce power consumption to at least 50% less than a comparable GPU-based inference setup.

These goals ensure that the final accelerator is not only functionally correct and efficient, but also relevant for real-world deployment. To model a real-world resource constrained platform, the Zybo Z7 was selected as the target FPGA. A practical use-case where these requirements intersect is automatic license plate recognition (ALPR), particularly in smart city and tolling systems. In regions like the UAE, where digit-only license plates are common and monitored at high frequency (e.g., via roadside or overhead cameras), the ability to process individual digits in real time is essential. Running digit classification at 30 fps enables the system to operate seamlessly in live traffic streams, while the energy savings of FPGA-based inference reduce operational costs and thermal overhead in embedded deployments.

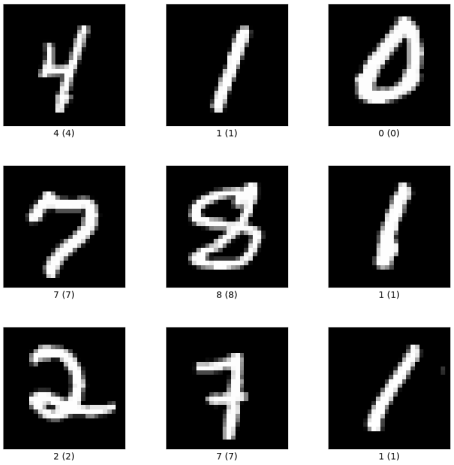

Figure 1.2: (a) MNIST handwritten digits

Figure 1.3: (b) UAE license plate (digit-focused)

Figure 1.4: Illustration of the MNIST dataset and a suitable application; a vehicle license plate.

Thesis Summary

Introduction: Introduces the energy challenges of AI inference and motivates FPGAs as low-power accelerators for CNNs, using ResNet18 and MNIST as a case study.
Literature Review: Explores the evolution of CNN architectures and FPGA implementations, highlighting ResNet18's balance of accuracy and hardware efficiency.
Dataset Justification: Discusses the use of MNIST and related digit datasets as suitable benchmarks for real-time, embedded vision tasks like ALPR.
Methodology: Details the adaptation of ResNet18 through quantisation-aware training, architectural simplification, and layer fusion to enable HLS-based FPGA deployment.
FPGA Design: Describes the modular development of synthesizable C++ IP cores with AXI4-Stream interfaces and custom buffering strategies for convolution.
System Integration: Explains how IP cores were composed into a complete pipelined inference pipeline using block design containers and streaming control logic.
Evaluation: Benchmarks the FPGA design against GPU inference in terms of accuracy, latency, power, and throughput, confirming its suitability for edge AI.
Conclusion: Summarises key findings and outlines future work including design automation, dataset scaling, and system-level parallelism.

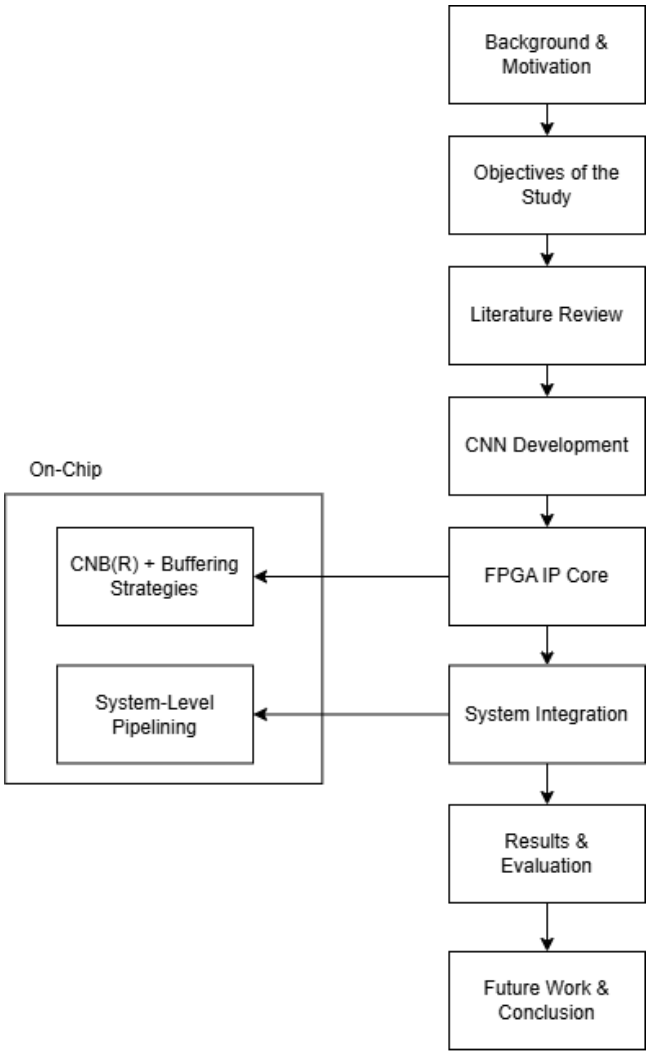

Figure 1.5: Thesis structure and implementation flow.

Chapter 2

Theory and Literature Review

2.1 Introduction to Deep Neural Networks (DNNs)

Deep Neural Networks (DNNs) are computational models inspired by the human brain's structure. These models comprise layers of interconnected "neurons" that transform inputs into increasingly abstract representations, enabling applications such as image recognition and speech processing [**?**,2]. Specialized variants of DNNs have emerged for distinct data types: Recurrent Neural Networks (RNNs) for sequential data, Transformer-based architectures for language processing, and Graph Neural Networks (GNNs) for graph-structured inputs [3].

Convolutional Neural Networks (CNNs) have become dominant in image classification tasks [3,4]. CNNs utilize convolutional layers to extract spatial hierarchies in data. As stated in [5], features like edges and shapes are captured efficiently while minimizing computation via local connectivity and parameter sharing. This makes CNNs highly effective for image tasks such as object detection and segmentation [3].

2.2 CNN Architectures and Dataset Justification

The development of CNNs has progressed from LeNet [6] to AlexNet [5], VGG [7], GoogleNet [7], and ResNet [8]. Each step aimed to improve accuracy, scalability, and computational feasibility. AlexNet introduced ReLU and dropout; VGG used uniform filter sizes; GoogleNet added inception modules; and ResNet, with skip connections, overcame degradation, a phenomenon where increasing model complexity resulted in reduced performance despite theoretically being able to represent more complex functions, in deeper networks [8, 9]. The implementation of a skip layer via the Residual Block allows gradients to flow through the network without vanishing, allowing the model to "decide" if it needs to learn a new representation or simply pass the input through.

ResNet18 is particularly well-suited for this project due to its balanced trade-off between complexity, modularity, and performance. It delivers strong accuracy on image classification tasks while maintaining computational efficiency [4, 10]. Importantly, it retains the same core building blocks, convolutional, pooling, activation, and skip connections, as found not only in deeper ResNet variants like ResNet50, but also across other modern CNN architectures, making it an effective proof of

concept for scalable deep learning designs. With modularity conscious hardware development, the reuse and reconfiguration of IP cores to support varying network depths, variants and architectures becomes possible. This approach not only optimizes inference on limited hardware but also lays the groundwork for deploying more complex CNN models on higher-resource platforms to tackle more demanding classification tasks [11].

The MNIST dataset [12] was selected for its simplicity and suitability for hardware prototyping. Comprising 28×28 grayscale images of handwritten digits, it enables rapid testing of feature extraction, quantization, and low-power deployment strategies [3, 4]. Though limited in complexity, MNIST aligns well with this project's ALPR use-case, where digit recognition is central. Its low-resolution format and classification scope make it an ideal fit for the resource constraints of entry-level FPGA platforms, unlike heavier benchmarks such as CIFAR or ImageNet.

2.3 Platform Suitability and Hardware Selection Rationale

Post-model development, the choice of deployment hardware is critical for achieving real-time, low-power performance, especially in edge AI applications. Edge AI refers to the execution of artificial intelligence models directly on local devices, such as sensors, cameras, or embedded platforms, rather than in centralized cloud servers. This approach reduces latency, limits bandwidth usage, and improves privacy, making it ideal for time-sensitive or resource-constrained scenarios.

While GPUs are the standard for training and high-throughput inference, their architecture is not optimized for latency-sensitive or power-constrained deployments. Designed primarily for large-batch parallel workloads, GPUs incur high static and dynamic power costs, leading to inefficiencies in small-batch inference [13]. This poses limitations in embedded and real-time applications, where responsiveness and energy efficiency are essential.

In contrast, Field-Programmable Gate Arrays (FPGAs) offer reconfigurable hardware that can be tailored to the specific structure of CNN workloads. They support fine-grained parallelism, pipelined execution, and deterministic latency, key advantages for real-time classification [14, 15]. Their flexible on-chip memory architecture minimizes costly off-chip data movement, enhancing throughput and reducing energy consumption. Notably, FPGAs allow for meticulous Register-Transfer Level (RTL) control, enabling deliberate management of dataflow between processing elements. This precise control results in more efficient hardware utilization and execution compared to general-purpose GPU architectures.

In comparative benchmarks, FPGAs achieved 5ms latency at 30W power consumption for a specific CNN task, compared to 8ms at 150W on GPUs [16]. These benefits are reflected in real-world deployments. For instance, Alibaba's FPGA-based inference system running ResNet18 demonstrated significantly lower latency and higher throughput at low batch sizes compared to GPU equivalents (Figure 1.1) [1], highlighting the advantage of FPGA-based acceleration in streamed, small-batch scenarios.

Beyond performance, FPGAs support rapid prototyping of task-specific accelerators, such as convolutional and residual blocks [14]. Once validated, these designs can be scaled to higher-end

FPGAs or ported to ASICs for production deployment. This design flow ensures that early development efforts remain viable in later stages.Additionally, FPGAs allow for domain-specific optimizations like custom memory hierarchies, quantization-aware execution, and specialized control logic.

The ability to reprogram hardware in response to dynamic workloads or operating conditions is especially useful in edge environments, where inputs and constraints frequently change. Combined, these features make FPGAs exceptionally well-suited for efficient and responsive deployment of CNNs like ResNet18 in real-time, low-power edge AI systems.

2.4 Hardware-Optimized CNN Design and HLS

Quantisation plays a critical role in adapting deep neural networks for deployment on resource-constrained platforms such as FPGAs [17]. By reducing the bit-width of weights and activations, quantisation significantly lowers memory usage, power consumption, and computational overhead, enabling real-time inference within tight area and energy budgets. Prior work has shown that quantisation can reduce resource usage by over 50% while preserving accuracy [18], making it a foundational optimisation for embedded AI.

Two common approaches exist: Post-Training Quantisation (PTQ), which is applied after training, and Quantisation-Aware Training (QAT), which simulates quantisation effects during training. PTQ enables faster deployment but often degrades accuracy at low bit-widths. QAT, by contrast, preserves model performance by applying quantisation during the forward pass while maintaining full-precision gradients during backpropagation. In this project, QAT was selected from the outset to ensure resilience to quantisation noise. An 8-bit version of ResNet18 was developed accordingly, balancing accuracy and hardware simplicity with mixed precision intrinsic executions. Energy efficiency improvements of up to 6× versus high-end GPUs have been reported for comparably quantised networks [13].

Given the limited resources of the target FPGA board, further compression and optimisation techniques were explored. Layer fusion, as proposed by Alwani et al. [19], was used to minimise memory traffic by fusing adjacent layers and reducing off-chip feature map transfers by up to 95%. This promotes data locality and reduces bandwidth bottlenecks. In parallel, channel compression, following the work of Liang et al. [20], was employed to prune redundant feature channels. This reduces computational load and supports pipeline simplicity without increasing parameter count, an important consideration in low-resource designs.

To implement this architecture efficiently, the project adopts a streaming, pipelined execution model inspired by AoCStream [21], where intermediate activations are passed between layers through local buffers rather than written to global memory. While AoCStream realises this in RTL, this project uses High-Level Synthesis (HLS) via Xilinx Vitis HLS to achieve similar behaviour. Convolutional modules were constructed using `hls::stream` objects alongside synthesis directives such as `#pragma HLS PIPELINE`, `ARRAY_PARTITION`, and `DATAFLOW` to control timing, parallelism, and memory mapping. These abstractions replicate RTL-style streaming while accelerating development.

HLS enables rapid iteration, platform retargeting, and behavioural abstraction by automatically generating FSMs and datapaths from high-level C/C++ descriptions [22]. However, this comes at a cost: HLS often produces designs with lower Quality of Results (QoR) compared to hand-crafted RTL, especially in timing closure, area efficiency, and arithmetic precision [22]. These limitations may have contributed to small degradations in accuracy and throughput in this project, an issue further examined in the evaluation.

2.5 Review of Notable Works

Numerous FPGA-based CNN accelerators have been proposed to address the challenges of deploying deep learning models on low-power, embedded platforms. These works vary in their focus, ranging from hardware-level optimization and modular IP core design to quantisation techniques and energy efficiency, yet share common goals of enhancing inference performance and resource utilization.

Several studies [23–25] implemented custom CNNs for handwritten digit recognition using modular and reconfigurable IP cores. Giardino et al. [23] utilized a 9-bit fixed-point representation on a Xilinx Artix 7 FPGA, achieving 90% accuracy while minimizing DSP usage. Xiao et al. [24] employed RTL-level reconfigurable modules with a dual-cache strategy on an Intel Cyclone10 FPGA, reaching 97.57% accuracy with sub-millisecond inference latency. Similarly, Zhu et al. [25] used HLS-generated IPs adaptable to kernel and stride sizes, deploying an 8-layer CNN with 98.89% accuracy and efficient modularity on Zynq-7020.

Parra et al. [26] presented an end-to-end methodology for deploying TensorFlow Lite (TFLite) quantised models on FPGAs, using a custom accelerator architecture compatible with AXI-Lite interfaces. Their implementation on the Zybo Z7 demonstrated competitive accuracy and a significant reduction in power consumption over CPU baselines, highlighting the practical benefits of integrating ML frameworks with FPGA inference.

Feng et al. [27] contributed an energy-efficient LeNet-5 accelerator using Vivado HLS with fixed-point arithmetic and approximated activation functions. Their design outperformed GPU counterparts in both energy and throughput, aided by loop pipelining and memory access optimization.

Finally, Biswal et al. [28] evaluated CNN, QNN, and BNN implementations on the Pynq Z2 board. Their comparative analysis revealed that CNNs provided a competitive balance between performance and efficiency, particularly for IoT-oriented applications requiring low-power inference.

Together, these works underscore the potential of FPGA platforms for efficient CNN deployment, demonstrating trade-offs in accuracy, latency, and hardware complexity while offering diverse paths to architectural optimization.

2.6 Research Gaps and Limitations

Despite substantial advancements in FPGA-based CNN accelerators, a significant research gap remains in the development of modular, scalable architectures that facilitate rapid prototyping, training, and deployment of complex models. Many existing works focus on lightweight or custom CNNs

tailored specifically for the MNIST dataset [23, 27], limiting their generalizability and reusability for more demanding applications. While some implementations achieve impressive accuracy or efficiency, they often rely on static, hand-crafted hardware configurations that are tightly coupled to a specific network structure, making adaptation or extension cumbersome [24, 25].

Moreover, the integration between quantised training pipelines and hardware deployment is often fragmented. Quantisation-aware training is rarely coupled with corresponding hardware design strategies, and design workflows lack cohesion between software-level model experimentation and hardware-level implementation [26, 28]. This disconnect hampers the iterative design and testing of more advanced architectures, such as residual networks, in embedded contexts.

This dissertation addresses this gap by presenting a modular, pipeline-oriented framework for deploying a quantised ResNet18 model on the Zybo Z7 FPGA. The approach emphasizes reusability and architectural clarity through the development of high-level synthesis IP cores that can be composed, scaled, and adapted with minimal overhead. By aligning the training, quantisation, and hardware design processes, this work enables faster iteration and deployment of complex CNN models, bridging a critical divide between machine learning development and embedded inference systems.

Chapter 3

Methodology

3.1 CNN Model Development and Training

The convolutional neural network (CNN) model was developed with a primary emphasis on achieving a balance between architectural complexity, computational efficiency, and deployment feasibility on FPGA hardware. The design process began with a ResNet34 implementation sourced from PyTorch repositories, which was then adapted into a ResNet18 configuration by removing selected layers. This reduction maintained the residual block architecture, ensuring that the model preserved its capacity for hierarchical feature extraction, while significantly lowering computational demands to suit the MNIST-scale classification task and the constraints of the target FPGA platform.

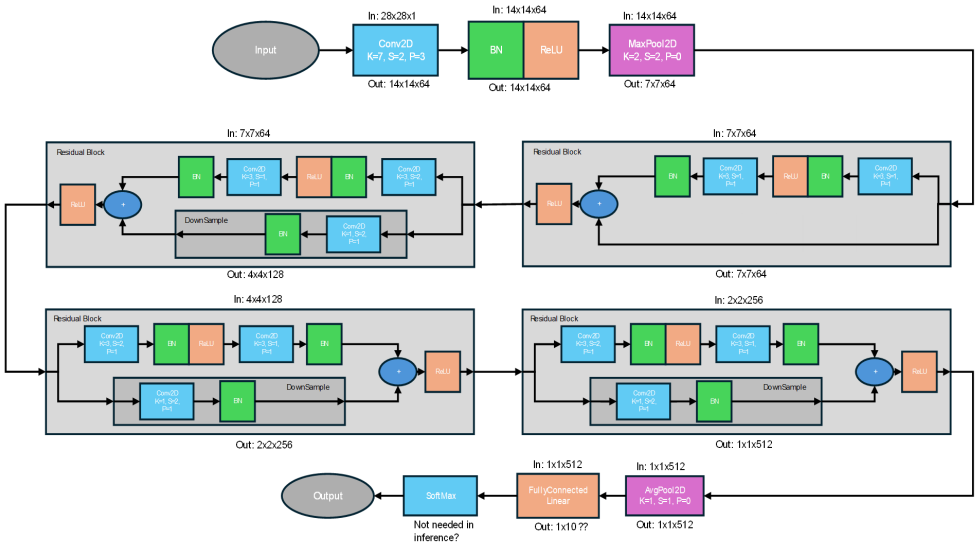

Figure 3.1: ResNet18 model adapted for MNIST.

Training was performed using stochastic gradient descent (SGD) with a momentum of 0.9 and an initial learning rate of 0.01. A cosine annealing learning rate scheduler was employed over 20 epochs to gradually reduce the learning rate and facilitate stable convergence. The batch size was set to 128 to optimize GPU throughput while maintaining memory efficiency, and training was carried out on GPU-accelerated hardware when available. The CrossEntropy loss function was used to optimize classification performance throughout the training process.

Quantisation-Aware Training (QAT) was implemented to prepare the model for reduced-precision inference on the FPGA platform. The quantised ResNet18 was trained using PyTorch's FX Graph Mode with the `fbgemm` backend, enabling the integration of fake quantisation for both weights and activations during training. This approach allowed the model to learn in the presence of quantisation noise and thus adapt its internal representations accordingly. Static quantisation parameters, including fixed zero points and scaling factors, were introduced during training, aligning the model for efficient integer-based execution. While static quantisation reduces the model's dynamic range compared to dynamic methods, it offers lower computational overhead and improved compatibility with hardware-accelerated inference, particularly for datasets like MNIST where input variability is limited.

Architectural optimisations were introduced after initial training to further enhance hardware compatibility. A critical step involved the fusion of convolution, batch normalisation, and ReLU layers into single composite operations. This fusion strategy, illustrated in Figure 3.4, was employed to improve data locality and reduce external memory traffic, thereby minimizing latency and energy consumption during inference. Although layer fusion limited some design modularity and flexibility, the benefits in terms of communication efficiency and resource usage made it an essential transformation for real-time FPGA deployment.

Further quantisation refinements were evaluated to balance accuracy with hardware constraints. Tensor-wise quantisation was chosen over channel-wise alternatives due to its reduced memory footprint and simpler scaling scheme, which are well suited to the resource limitations of embedded FPGA systems. Although this method sacrifices some granularity in representing activation distributions, empirical evaluations indicated negligible performance degradation on MNIST. The comparison of quantisation strategies is presented in Figure 3.3.

To reduce the model's memory and computational footprint further, the number of channels in convolutional layers was reduced by a fixed factor. A detailed evaluation of accuracy versus simplification ratio, shown in Figure 3.2, led to the selection of a 16× simplification factor. This adjustment yielded substantial resource savings while maintaining high classification accuracy, ensuring that the final design remained within the target FPGA's processing and memory budgets.

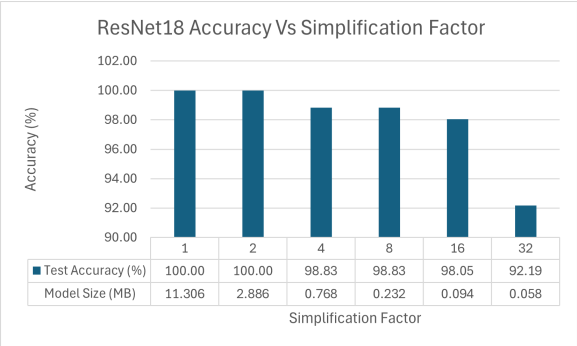

Figure 3.2: Evaluation of accuracy versus channel simplification factor. A 16× simplification factor was selected based on this analysis.

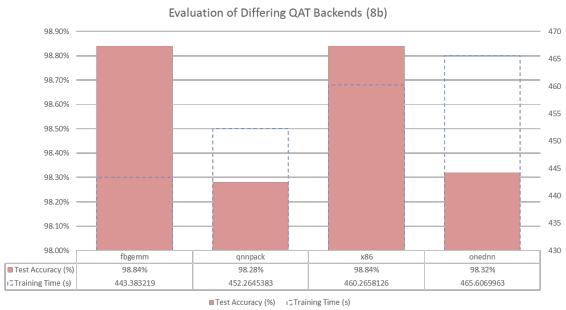

Figure 3.3: Comparison of quantisation-Aware Training methods: tensor-wise versus channel-wise quantisation. Tensor-wise was selected for its superior memory efficiency.

The final architecture, depicted in Figure 3.4, integrates all these optimisations to produce a

model that is both functionally accurate and highly efficient for deployment in FPGA-based systems. Validation through CPU-based simulation confirmed the correctness and performance equivalence of the quantised model compared to its floating-point counterpart. This outcome demonstrates the effectiveness of the applied training regime and structural refinements in achieving an optimal trade-off between model performance and hardware feasibility.

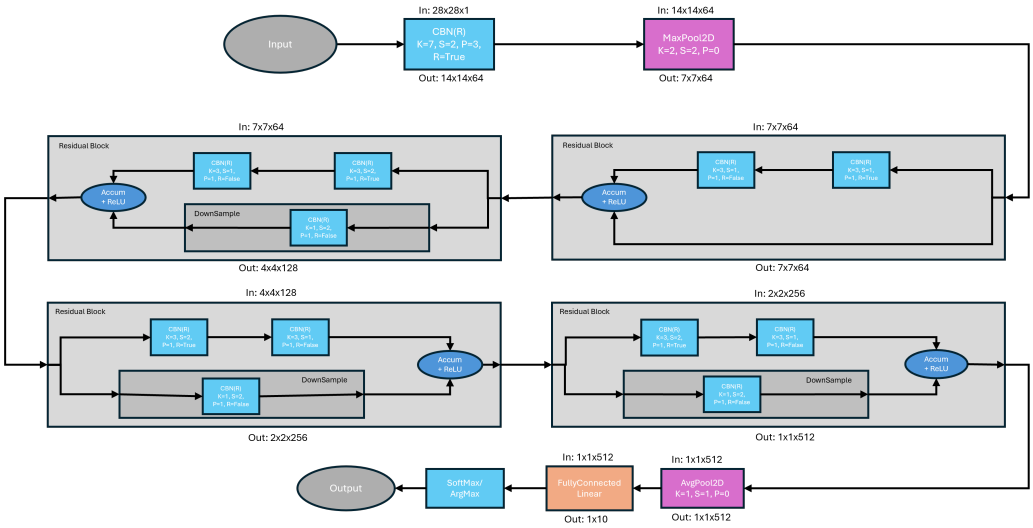

Figure 3.4: Hardware-suitable ResNet18 architecture with fused layers for FPGA execution.

3.2 FPGA Platform and Tools

The FPGA implementation was deployed on the Xilinx ZyboZ7, which features a Zynq-7000 SoC combining an ARM Cortex-A9 processing system with a mid-range Artix-7 FPGA fabric. The ZyboZ7 was selected for its accessibility, cost-effectiveness, and representative resource constraints, making it a suitable platform for evaluating real-time CNN inference in embedded systems.

High-Level Synthesis (HLS) was conducted using Vitis IDE 2024.2, enabling hardware description in synthesizable C++. Each core CNN operation, convolution, activation, pooling, and softmax, was developed as a separate module and synthesised into a reusable IP core. It was selected for its improved user interface simplification and the availability of more recent tutorials and resources, despite its inferior HLS synthesis compared to earlier versions. The Vivado 2024.2 development environment was used for system integration and deployment, selected for IP export compatibility with Vitis 2024.2.

PyTorch was chosen over TensorFlow for CNN model development due to its resource efficiency and better integration with the hardware-specific workflows. Models trained in PyTorch were converted to a format compatible with the FPGA implementation.

The design flow was primarily manual. IP core generation, integration, and system-level packaging were performed through the Vivado GUI and TCL scripts, but full end-to-end automation was not implemented in this version. Future iterations may benefit from a scripted or containerised workflow to improve design reproducibility and testing efficiency.

Table 3.1: Zybo Z7-20 Resource Metrics

Resource	Specification
FPGA Model	Xilinx Zynq-7000 (XC7Z020)
LUTs	53,200
FFs (Flip-Flops)	106,400
DSP Slices	220
BRAM (Block RAM)	560 KB (140 × 36 Kb)
Clock Frequency	Up to 650 MHz (PL), 766 MHz (PS)
I/O Pins	200 (approx., depending on bank usage)
On-chip Memory	512 MB DDR3 (external)
Processing System	ARM Cortex-A9 Dual Core

3.3 FPGA IP Core Design

The FPGA IP core development process followed a structured flow, beginning with a high-level C-based model, functionally verified against PyTorch outputs for correctness. This model was then refactored into synthesizable C++ with architectural efficiency in mind. Each neural network layer was developed as a standalone module to support independent synthesis, targeted optimisation, and modular validation. Reusable hardware primitives, such as window buffers, activation functions, and stream adapters, were abstracted into common components to streamline integration and enhance code maintainability.

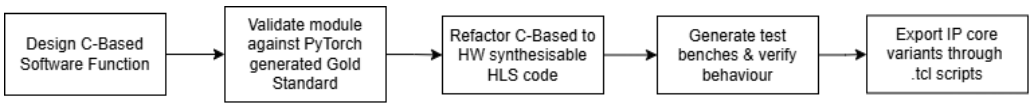

Figure 3.5: Flowchart depicting layer-wise IP core development flow.

Test benches were constructed using representative data drawn from the trained model's activation distribution. These enabled accurate simulation of dynamic range, saturation behaviour, and overflow, which in turn informed bit-width allocation for internal accumulators and intermediate datapaths. Where necessary, accumulator widths were widened to prevent overflow while avoiding unnecessary use of logic resources.

All inter-module communication was implemented using $hls :: stream$ constructs, forming implicit FIFOs between modules. This abstraction reduced the need for explicit buffering, simplified communication patterns, and supported pipelined execution with minimal latency.

To optimise performance, key HLS pragmas were applied throughout the design. These compiler directives influence how high-level C/C++ code is translated into hardware logic without altering its functional behaviour. All major synthesis directives, such as loop pipelining and memory partitioning, were applied at the top-level to preserve modular clarity and provide explicit control over performance-critical regions.

Specifically, '#pragma HLS PIPELINE' was used to allow loop iterations to initiate every clock cycle, achieving initiation interval (II) = 1 in core compute loops. Likewise, '#pragma HLS ARRAY_PARTITION' was used to split coefficient and buffer arrays along the unrolled dimension,

enabling concurrent access and eliminating memory bottlenecks. These optimisations were essential in achieving high-throughput, low-latency execution without relying on manual RTL development.

3.3.1 Convolutional Layer (CBN(R))

A convolution function is defined as a dot product between a kernel and a region of the input feature map. The convolutional layer is the core component of CNNs, responsible for extracting local features from the input data. The convolution operation is defined mathematically as:

$$O(x, y) = \sum_{i=0}^{IC-1} \sum_{j=0}^{KH-1} \sum_{k=0}^{KW-1} I_i(x + j, y + k) \cdot W_{o,i}(j, k) \tag{3.1}$$

Code-suitable implementations of the convolutional operations can be found in Appendix A. The base algorithm, algorithm 1 takes an input feature map, a kernel, and a bias term as inputs. It iterates over the output dimensions, applying the kernel to the corresponding region of the input feature map and accumulating the results. The output is then returned as a new feature map. The algorithm incorporates two key features: stride and output-side padding. Stride allows the kernel to move across the input feature map with a specified step size, while output-side padding ensures that the output dimensions are consistent with the expected size.

The convolutional IP core was implemented using a streamed, output-unrolled accumulation architecture. Once the input line and window buffers were primed, each incoming pixel was processed per clock cycle. The line buffer held a rolling set of input rows, while the window buffer extracted the active kernel region required for the dot product. This enabled real-time processing with high data reuse and eliminated the need for external memory accesses. The buffering strategy is illustrated in Figure 3.6.

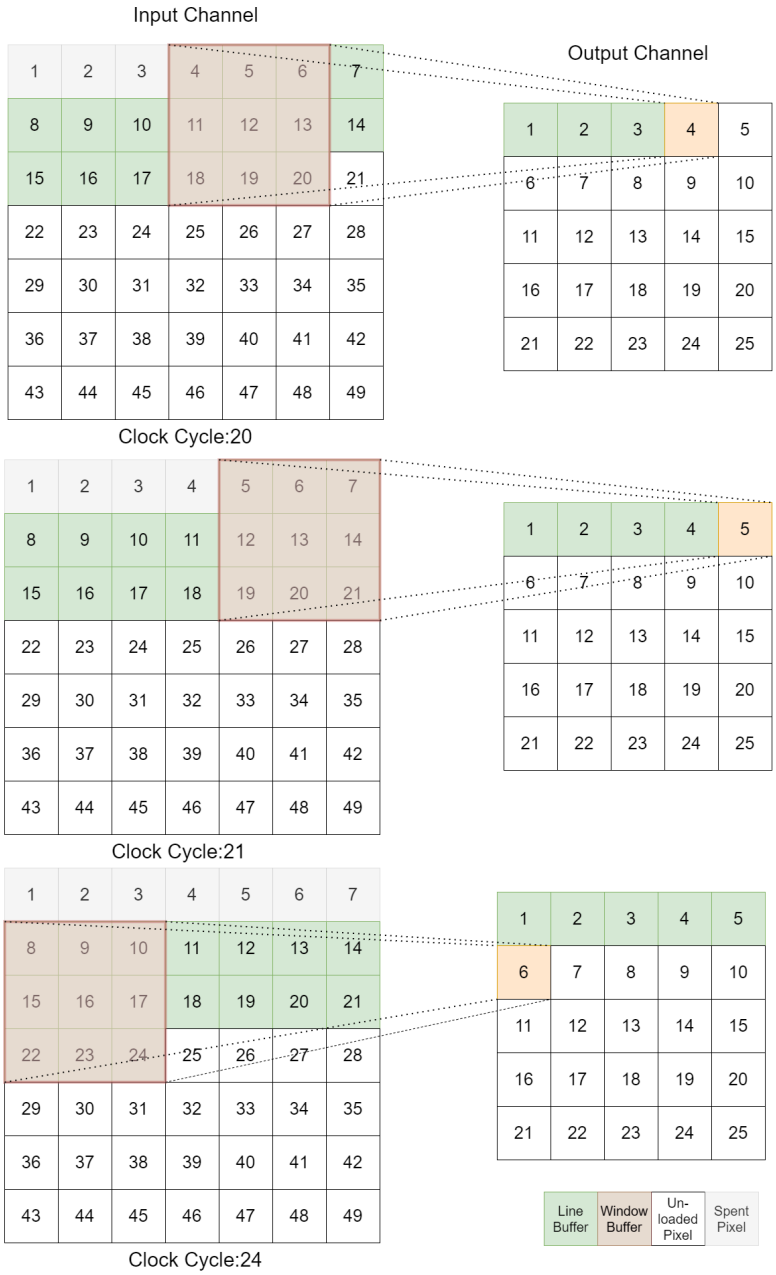

Figure 3.6: Line and window buffering in convolution.
Line buffer size: $(W \cdot KH - 1) + KW$. Window buffer size: $KH \cdot KW$.

Two algorithms were pitted against eachother: the output-first algorithm (Alg. 2), which processes one output channel at a time, and the some-at-once algorithm (Alg. 3), which processes multiple output channels in parallel throughout kernel-unrolling. The output-first algorithm is simpler to implement and requires less buffering, while the some-at-once algorithm can achieve higher throughput by processing multiple output channels simultaneously. The non-linear resource usage scaling due to varying extents of reuse was used to unroll differing layers to varying degrees to minimise the greatest latency layer.

Table 3.2: Comparison of output-first vs. partially unrolled convolution implementations.

Approach	Latency (cycles)	BRAMs	DSPs	FFs	LUTs
Output-First (UF=1)	3,276	0	8	910	1798
Output Partially Unrolled (UF=4)	840	1	26	1638	3236

IC = 4, OC = 4, Input = 28×28, Kernel = 3×3

Weights were stored in BRAM, with a partitioning strategy that allowed each unrolled compute unit to access its respective weights without read conflicts. The accumulation itself exploited the associative and commutative properties of addition, enabling flexible scheduling without altering correctness, and allowing for simple BRAM weight packing, packing all weights to 8b for read simplification. Although reordering coefficients to improve numerical precision, such as pairing large and small weights to enhance bitsliced multiplier efficiency, was theoretically explored, limitations in Vivado HLS prevented dynamic bit-level packing strategies from being realised.

A hybrid strategy using varying unroll factors (UFs) was adopted to balance resource usage and throughput. This allowed multiple output channels to be computed in parallel while keeping resource utilisation within Zybo Z7 constraints. Array partitioning pragmas were applied to output channels and kernel loops to unlock parallelism, the coefficient and output arrays were partitioned to the same degree as the unroll factor (UF), optimised on a per-layer basis to minimise latency of the greatest latency layer. Inner loops over input channels were pipelined with initiation interval (II) = 1, executing one operation per cycle. Multi-cycle operations were attempted, without success due to HLS complications; further exploration of this approach may yield better results in future iterations.

All arithmetic was performed in 8-bit signed fixed-point format. Batch normalisation parameters were statically folded into the convolution weights and biases prior to deployment. ReLU activation was implemented inline via conditional clamping immediately after accumulation. Accumulators were widened to varying precision depths, optimised to prevent overflow and clamped prior to streaming. Optimal precision was determined through passing a ”white image” (full 255 pixels) to the actual trained weights and determining the saturation % for each precision, a maximum saturation % of 0.01% was supported. Padding was handled during output write using conditional logic, avoiding the timing complexity that arises with pipelined read-side padding.

3.3.2 Pooling Layer

Pooling layers were implemented to reduce spatial resolution while preserving key features from the input activation maps. Two types of pooling were considered: max pooling and average pooling.

Mathematically, for an input feature map I and a pooling window of size $K \times K$, located at position (x, y), the pooling outputs are defined as:

$$\text{MaxPool}(x, y) = \max_{0 \le i, j < K} (I(x + i, y + j)) \tag{3.2}$$

$$\text{AvgPool}(x, y) = \frac{1}{K^2} \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} I(x + i, y + j) \tag{3.3}$$

Both pooling modes reused the convolutional line buffer and window extraction infrastructure. The input stream was passed through a line buffer, from which the active $K \times K$ pooling window was extracted. For max pooling, a pipelined comparator tree was used to compute the maximum value in each window. The entire pipeline was fully pipelined and did not require any multipliers, operating efficiently under the fixed-point arithmetic format used throughout the design.

Average pooling followed a similar structure, replacing the comparator tree with an adder tree and applying a final division by K^2. The pooling layer supported parameterisation for kernel size, stride, and pooling type (max or average), allowing it to be configured per layer if needed.

3.3.3 Final Connected Layer

The FC layer computed a dot product between the flattened input and each weight vector. Biases were initialised in a pipelined loop, followed by a pipelined accumulation loop for each output. Output channels were unrolled by a factor of 2 to boost parallelism.

$$(\text{Logit}) * o = b_o + \sum *i = 0^n W_(o, i) \cdot x_i \tag{3.4}$$

where o is the output channel, b is the bias, W is the weight vector, and x is the input vector. The logit was computed in 8-bit fixed-point format, with a 16-bit accumulator to prevent overflow. The final output was clamped to 8 bits before being sent to the softmax layer.

3.3.4 Softmax/Argmax and Output Logic

Softmax is needed for training forward pass inference applications, for final deployment classification uses Argmax is sufficient. The softmax layer was implemented as a separate module to allow for flexible deployment options. The softmax function is defined as:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=0}^{C-1} e^{x_i}} \tag{3.5}$$

where x_i is the logit for class i, and C is the number of classes. The softmax function normalises the logits into a probability distribution over the classes.

Softmax transformed logits into probabilities using 32-bit fixed-point arithmetic for accuracy. Inputs were shifted by their maximum to prevent overflow. Exponentiation was approximated via LUTs, and normalisation was performed using reciprocal approximation and multiplication.

Though softmax consumes more resources, it runs only once per image and contributes 5% of total usage with just 47 cycles of latency. For applications needing only a class prediction, softmax can be replaced with an argmax comparator, reducing latency to just 19 cycles and resource usage to virtually 0% of the total. This is particularly useful for applications where only the most likely class is needed, such as in real-time classification tasks. The softmax layer was designed to be flexible, allowing for both softmax and argmax modes of operation. The choice between these modes was made at runtime based on the application requirements.

Output probabilities were downscaled to 8-bit and sent via AXI-Lite to the host for post-processing. This softmax design provides accurate, deterministic inference with minimal system overhead.

3.4 System Integration

Two architectural strategies were considered during system integration: a single shared compute engine reused across all layers, and a parallel processing element (PE) approach with dedicated compute units for each layer. While the shared engine offered compactness, it incurred high latency due to the need to reconfigure memory and control paths between layers and the inherent serialisation of operations, which prevented concurrent execution and led to significant idle time between processing stages. The parallel PE model, though more resource intensive, enabled concurrent layer-specific optimisations and significantly reduced total execution time. Given the differing layer sizes within ResNet18, the parallel PE approach proved more efficient, as it supported tailored unroll factors and buffer allocations per layer, fully exploiting the streaming nature of the workload. Additionally, static buffer sizing permitted by independent IP cores eliminated dynamic memory overhead and enabled compile-time buffer optimisation, improving overall resource utilisation.

All layers were exported as unique, parameterised IP modules using an automated `.tcl` script to accelerate development and minimise unused overheads. A fully parallel implementation exceeded the available resource budget; therefore, a hybrid strategy, moderate parallelism combined with sequential execution, was adopted. This configuration, guided by synthesis feedback and profiling, ensured an optimal balance between performance and feasibility, allowing successful deployment on the Zybo Z7 platform.

Residual blocks were developed and validated independently in Vivado Block Design, then integrated into the final architecture using Block Design Containers (BDCs). This modular structure allowed for incremental testing and simplified debugging. Once assembled, the entire BDC system was synthesised and implemented as a unified design, ensuring correct inter-block connectivity and compliance with timing constraints.

Convolutional layers were designed for pipelined execution, allowing the output of one layer to stream directly into the next. This eliminated the need for intermediate memory buffers and significantly reduced memory bandwidth demands.

AXI4-Stream interfaces were used for inter-layer communication, enabling continuous dataflow between CNN blocks without intermediate memory buffering. Each module began processing data as soon as it became available, allowing pipelined execution and eliminating latency from memory reads or writes. The ARM processor acted as the AXI-Lite master, configuring IP cores and managing control signals, while weights, biases, and convolutional kernel coefficients were preloaded into BRAM for low-latency access near the compute fabric.

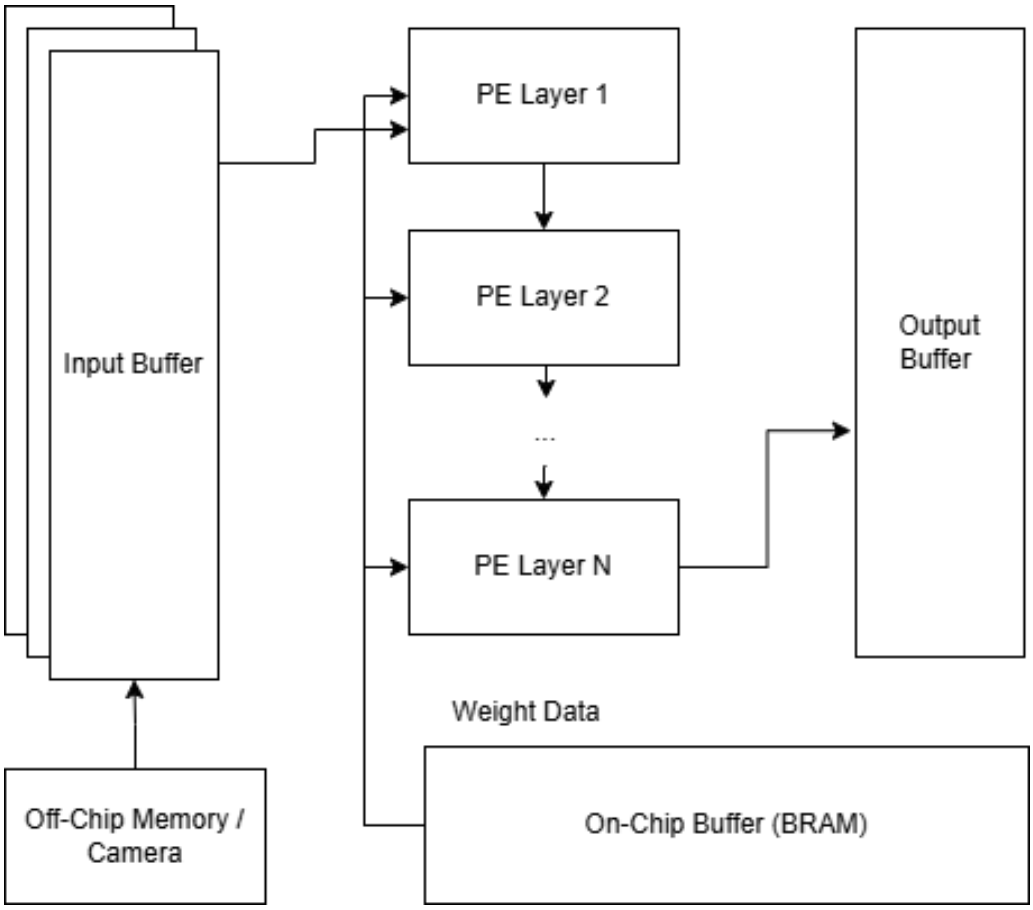

Figure 3.7: System-level architecture showing pipelined convolutional blocks and AXI interface connections.

Intermediate buffers are needed to store values for residual skip connections.

In summary, the integration approach combined architectural flexibility with effective resource management. The adopted parallel PE strategy delivered high throughput and low latency, validating its suitability for embedded real-time inference on resource-constrained FPGA platforms.

Chapter 4

Results and Evaluation

To establish a baseline for comparison, the performance of ResNet18 inference on the MNIST dataset was evaluated using a GPU across varying batch sizes. This analysis highlights the scalability of GPU-based inference and frames the limitations that motivate FPGA acceleration in real-time edge scenarios.

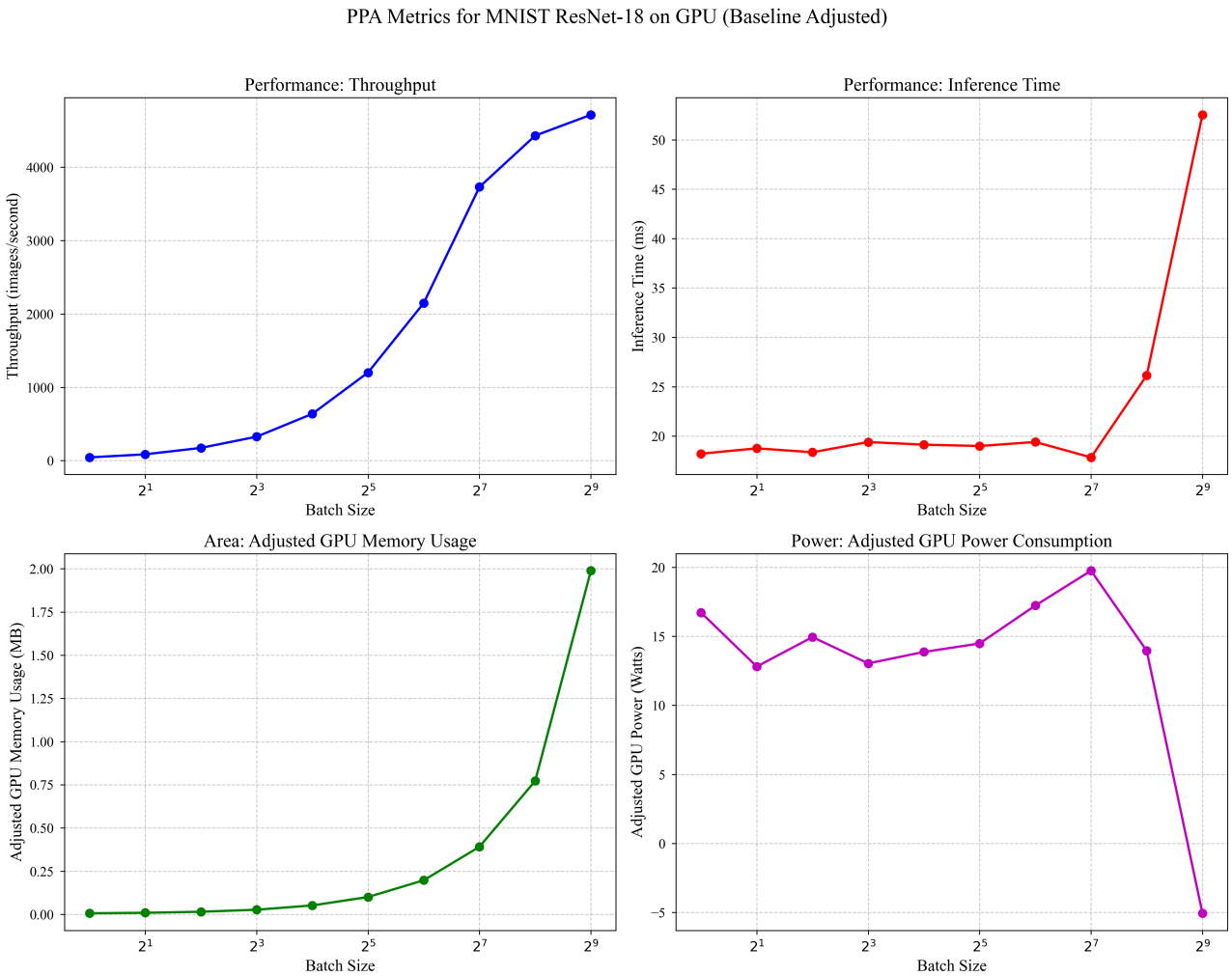

Figure 4.1: GPU PPA metrics for ResNet18 on MNIST across batch sizes.

As illustrated in Figure 4.1, GPU throughput (top-left) scales approximately linearly with batch

size due to the high degree of parallelism offered by modern GPU architectures. However, this trend begins to saturate beyond a batch size of $2^7 = 128$, where core and memory bandwidth limitations begin to bottleneck further performance gains. This saturation is mirrored in the inference time plot (top-right), where latency remains relatively stable up to batch size 128, then rises sharply due to memory contention and increased queuing overhead.

The area-related metric (bottom-left) shows adjusted GPU memory usage rising sharply after batch size 64, indicating significant growth in memory allocation required for parallel processing. Similarly, adjusted power consumption (bottom-right) increases steadily with batch size, peaking at 128, after which it drops, likely due to backend throttling or stalled execution from resource saturation.

Despite these favorable trends at high batch sizes, GPU performance degrades significantly in low-latency, real-time settings. For this reason, all comparative evaluation in this work is conducted at a batch size of 1, which reflects the effective streaming condition of a 30fps inference system. Under this constraint, the GPU exhibits poor throughput, relatively high per-frame latency, and inefficient power usage, making it an ill-suited choice for embedded or edge deployment without batching. This highlights the importance of specialised hardware accelerators, such as FPGAs, for low-latency inference in resource-constrained environments.

Table 4.1: FPGA Synthesis Results Summary

Metric	**Value(%)**
Clock Frequency (MHz)	137.4
Latency (Cycles)	2422
Latency (ms)	17.63
Throughput (FPS)	56.73
LUTs Used	47556 (89.4)
BRAMs Used	22 (15.7)
FFs Used	25844 (24.3)
DSP Slices Used	208 (94.5)
Power Consumption (W)	1.43
Power Efficiency (FPS/W)	39.67

Table 4.2: Platform Comparison: FPGA vs. GPU for ResNet18 on MNIST

Resource	**FPGA**	**GPU**
Power (W)	1.43	33.21
Latency (ms)	17.63	18.21
FPS	56.73	44.26
Power Efficiency (FPS/W)	39.67	1.33
Accuracy (%)	98.7	98.9

Both platforms are executing model in Figure 3.4 on MNIST at BS=1.

The results of the synthesised FPGA implementation are presented in Table 4.1, while a direct comparison with GPU baseline metrics is shown in Table 4.2. The implemented FPGA accelerator achieved a clock frequency of 137.4 MHz and sustained a throughput of 56.73 frames per second (FPS) at a latency of 17.63 ms per image, exceeding the 30 FPS real-time threshold defined in the project goals.

In terms of power, the FPGA consumed only 1.43 W under full load, significantly lower than the 33.21 W measured for the GPU. This results in a power efficiency of 39.67 FPS/W, which is approximately 30× higher than the GPU's 1.33 FPS/W. These results underscore the substantial energy advantage of FPGA-based inference in low-batch, real-time environments.

From a hardware utilisation perspective, the implementation remained within the bounds of the target FPGA, using 89.4% of LUTs, 15.7% of BRAM, 24.3% of flip-flops (FFs), and 94.5% of available DSP slices. While DSP usage approached the device limit, careful pipelining and unrolling strategies ensured timing closure and full functionality without overutilisation. Importantly, despite

resource constraints, the final model maintained a classification accuracy of 98.7%, nearly matching the GPU baseline of 98.9%.

This FPGA implementation clearly outperforms the GPU in the low-latency, streaming-inference regime evaluated at batch size 1. The GPU, while exhibiting higher performance at large batch sizes due to its fixed parallel compute grid, is fundamentally less efficient when operating in frame-wise inference mode due to overhead from scheduling, memory access latency, and unused parallel resources. The FPGA, by contrast, is optimised specifically for this execution regime, leveraging task-specific datapaths and a tightly pipelined architecture.

Most critically, the implemented design successfully meets, and in several cases exceeds, all target objectives outlined in the thesis:

- Achieved real-time throughput of 56.73 FPS (target: 30 FPS).
- Achieved inference accuracy of 98.7% (target: $\geq$ 95%).
- Achieved more than 25× better power efficiency than the GPU (target: $\geq$ 2×).
- Demonstrated complete deployment feasibility within resource constraints of a commercial FPGA platform.

These results validate the proposed hardware-aware optimisations and demonstrate the viability of FPGA-based convolutional inference for real-world, edge-class AI workloads. The implementation provides a compelling case for adopting task-specialised architectures over general-purpose accelerators in latency-critical and power-constrained environments.

A notable quirk in the results is the disparity in GPU and FPGA accuracy. This is due to the convolutional layer's precision being selected as a global parameter across all layers, rather than being optimised per layer. This simplified approach lead to saturation in a single layer with a few pixels not accurately extracting features, resulting in a slight drop in accuracy, particularly in the classification of the number 9.

While the implementation achieved its performance, accuracy, and efficiency targets, several oversights in the initial goal definitions became apparent during development. Most notably, the ResNet18 architecture was found to be excessive for the MNIST classification task, leading to sub-optimal resource-to-accuracy efficiency. Alternative, more lightweight models could have achieved similar or better classification accuracy on MNIST using fewer resources.

Additionally, the Zybo Z7 platform lacked sufficient resources to deploy the full ResNet18 architecture at its original depth. As a result, the network was significantly simplified, primarily through reduced channel widths and partial layer removal. Although these simplifications allowed the project to meet its defined objectives, they diverged from the original implementation plan.

Further performance limitations arose from constraints in High-Level Synthesis (HLS). Techniques such as output bitslicing and multi-cycle latency scheduling via `II=2+` pragmas often failed to yield the expected optimisations. These issues align with prior findings in the literature, where HLS-generated logic has been found to utilise 41% more resources than hand-written RTL implementations [22].

Finally, although the FPGA outperformed the GPU in real-time, single-image inference, the GPU still holds an advantage for high-throughput, batched workloads. With appropriate buffering and staggered image dispatch, GPUs can simulate real-time 30 FPS operation while leveraging their fixed parallelism more effectively. Thus, the superiority of the FPGA is context-dependent, particularly suited for strict latency and power-constrained scenarios, but less favorable in high-volume, parallel workloads.

These insights reinforce the importance of goal definition and platform selection in hardware-software co-design and suggest that future work may benefit from either targeting more complex datasets with ResNet18 or deploying more resource-efficient networks for MNIST-scale tasks.

4.1 Discussion

To contextualise the performance of the proposed accelerator, its results are evaluated against recent FPGA-based CNN deployments for MNIST classification, as shown in Table 4.3. Key comparisons are made in terms of classification accuracy, latency, resource utilisation, and power efficiency.

This implementation achieves a classification accuracy of 98.7%, placing it among the top performers. Unlike many prior works that employ task-specific or shallow architectures such as LeNet [27] or custom MLPs [23], this design retains the deeper structure and residual connections of ResNet18, providing a more realistic foundation for scalable deployment. While Feng et al. [27] report slightly higher accuracy (99.0%) with LeNet-5, their model is less representative of modern CNN workloads. The use of quantisation-aware training in this work contributes significantly to preserving accuracy despite reduced bit precision.

The system achieves a latency of 17.63 ms, equating to approximately 56 FPS. Although higher than that of Xiao et al. [24] and Feng et al. [27], both of which report sub-millisecond inference, those designs target minimalistic models implemented in hand-crafted RTL. By contrast, this work demonstrates real-time performance using a deeper model built entirely with Vivado HLS. Latency overheads are largely attributable to the toolchain's limited support for advanced scheduling, fine-grained pipelining, and multi-cycle operations.

Power consumption is a notable strength. At 1.43 W, the design offers significantly better efficiency than several comparable systems on the same Zynq-7020 platform, including Parra et al. [26] (4.5 W) and Biswal et al. [28] (2.54 W), both of which exhibit lower accuracy and higher latency. Giardino et al. [23] report lower power at 0.85 W, but with a much simpler architecture and a 90.0% accuracy ceiling. This demonstrates the effectiveness of model-aware hardware adaptation when energy constraints are critical.

Resource utilisation is moderate given the model's complexity: 47k LUTs, 22 BRAMs, and 208 DSPs. While more efficient designs exist, particularly those using shallow or custom networks, this implementation fits within the Zynq-7020 constraints while maintaining a realistic CNN topology. Some inefficiencies are attributable to the abstraction overhead of Vivado HLS. Attempts to implement bitsliced arithmetic, DSP reuse, and manual timing control were limited by the tool's inability to expose necessary low-level constructs such as bitfields or custom scheduling constraints. These limitations

are technical rather than architectural and represent a clear area for future improvement, potentially through RTL augmentation or alternative toolchains.

Importantly, this work differs from others by preserving the structure of ResNet18 rather than approximating it. This choice supports generalisation to more complex datasets and ensures that the design methodology remains applicable beyond MNIST. In doing so, it prioritises real-world applicability over benchmark optimisation.

Table 4.3: Comparison of FPGA-based CNN Inference on MNIST

Work	**Model**	**FPGA**	**Design Flow**	**Accuracy (%)**	**Latency (ms)**	**Resource Util. (%)**	**Power (W)**
This Work	ResNet18	Zynq-7020	HLS	98.7	17.63	47556/22/25844/208	1.43
[23]	Custom CNN	XC7A100T	RTL	90.0	41	15796/73/106400/0	0.85
[24]	Custom CNN	Cyclone10	RTL	97.54	0.0176	12588/-/48765/274	-
[27]	LeNet-5	Zynq-7000	HLS	99.0	0.151	73/66/26/43 *	3.32
[25]	Custom CNN	Zynq-7020	HLS	98.89	74	25004/25/34048/88	-
[26]	Custom CNN	Zynq-7020	-	94.43	127	6373/-/12470/93	4.5
[28]	Custom CNN	Zynq-7020	-	5.5	2.9	6373/-/12470/93	2.54

Note: Resource Utilisation is reported as LUTs/BRAMs/FFs/DSPs. Power consumption is reported as dynamic power.
*Reported as % since nominal values omitted.

In conclusion, the accelerator demonstrates strong accuracy, power efficiency, and real-time capability within a constrained FPGA and a high-level design flow. While latency and DSP packing could be improved through finer control or hardware upgrades, the system proves that deployment of deep, quantised CNNs is feasible using accessible tools and modest hardware. It offers not just a performance result, but a replicable workflow for practical edge AI acceleration.

Chapter 5

Conclusion and Future Work

This project demonstrates the successful deployment of an optimised ResNet18 CNN accelerator on the Zybo Z7 FPGA for real-time, low-power digit classification. Architectural simplifications, including quantisation, layer fusion, and depth reduction, ensured compatibility with the FPGA's resource limits. Alongside hardware-specific strategies such as line-buffered streaming, activation-aware accumulator clamping, and targeted loop unrolling and memory partitioning, the implementation exceeded the initial goals. These included achieving real-time throughput (56.73 FPS), high classification accuracy (98.7%), power efficiency (39.67 FPS/W), and complete deployability on the target platform.

While all objectives were met, several limitations were encountered, stemming not from architectural choices but from constraints within the High-Level Synthesis (HLS) toolchain. Optimisations such as bitsliced DSP packing and multi-cycle latency scheduling, though conceptually valid, were limited by Vitis HLS's lack of fine-grained bitfield manipulation and its rigid timing model. These issues likely result from the probabilistic synthesis framework of HLS tools, highlighting a broader challenge in FPGA-based ML acceleration: designer capabilities are often restricted more by the toolchain than by the hardware.

Another notable limitation lies in the pairing of dataset and model architecture. Although ResNet18 is a strong benchmark, it is unnecessarily complex for MNIST. Even after simplification, the model's depth and parameter count resulted in less efficient use of resources compared to lighter networks like LeNet or MobileNet. This mismatch complicates fair comparison with other FPGA accelerators. Future work should either use a more complex dataset such as CIFAR-10 to justify the model's capacity or adopt a simpler architecture better suited to MNIST. Aligning task complexity with model architecture and benchmarking practices is essential for meaningful evaluation.

Nevertheless, this work confirms the viability of resource-efficient edge AI inference. It shows that task-specific architectural optimisation allows FPGAs to deploy deep learning models effectively, achieving performance and energy efficiency competitive with GPU-based systems when both model and hardware are jointly optimised.

Although the architecture illustrated in Figure 3.4 met the design targets, it exceeded the complexity needed for high MNIST accuracy. To assess potential for further simplification, residual blocks were removed sequentially from the final stage, where latency savings are most impactful due to diminishing feature map sizes. Table 5.1 summarises these results, showing that notable reductions in latency

were achievable with minimal accuracy loss, underscoring a useful trade-off for future designs.

Table 5.1: Comparison of model simplification degrees and their impact on accuracy.

Model Type	Residual Blocks	Layers	Accuracy (%)	Training Epochs
ResNet18 (Adapted)	4	16	98.70	4
ResNet15-ish	3	13	98.44	4
ResNet12-ish	2	10	97.84	4
ResNet9-ish	1	7	94.53	4
No Skip Layer	0	6	86.74	20

Another avenue for future exploration is to adopt a simplified model to further reduce latency and power usage while maintaining acceptable accuracy. On the hardware side, improvements are also possible. The current design is constrained by the DSP count of the Zybo Z7, which limits parallel MAC operations. Migrating to a more capable FPGA platform, such as the ZCU102 or ZCU104, would enable greater parallelism and higher throughput.

One promising optimisation is multi-clock domain execution, shown in Figure A.1. By running memory access and compute stages at different clock frequencies (f_1 and f_2), it is possible to decouple high-speed input buffering from slower convolution operations. For example, line and window buffers can ingest data quickly, while accumulation runs at a lower frequency. This smoothing of bursty data traffic increases buffer utilisation. In the example shown, clock cycles 21–24 are idle due to data unavailability. More aggressive buffer filling and the use of clock domain crossing (CDC) logic can reduce these gaps.

Bitsliced arithmetic also offers considerable potential. Standard HLS synthesis maps one multiplication per DSP, but bitslicing enables multiple low-precision MACs, such as two 8-bit or three 6-bit operations, to share a single DSP. This improves MAC density and power efficiency. The fused output must then be unpacked using masking and shifting logic, as shown in Figure A.2. On the input side, BRAM-stored coefficients are packed based on the unrolling factor, with weight pairing balanced for DSP utilisation. Although such strategies were conceptually trialed, e.g., combining 8- and 7-bit weights, they could not be reliably synthesised in Vitis HLS due to bitfield control limitations. Nevertheless, previous work has proven the technique's value [29], making it a strong target for future implementation.

Further gains may be achieved through time-multiplexed DSP reuse. Downsampling branches often incur more latency due to stride and channel changes. By clocking these branches more slowly and reusing DSPs from faster identity paths, outputs can be aligned in time, improving utilisation and avoiding idle cycles or pipeline stalls without increasing the resource footprint.

Toolchain enhancements also present a valuable opportunity. HLS scheduling constraints could be addressed through hardware/software co-design within the Vitis framework. Techniques such as RTL augmentation, selective inlining, or the use of optimised IP cores may help mitigate timing bottlenecks. Evaluating newer versions of the toolchain or alternative synthesis flows may further expand the design space.

Finally, scaling to more complex datasets like CIFAR-10 or TinyImageNet would allow for testing generalisation and robustness. Alternatively, adopting compact architectures such as MobileNetV2

or SqueezeNet could support higher resolutions or more classes without exceeding hardware limits. This would also establish a more relevant benchmark for cross-comparisons and push performance-per-resource further.

On the deployment side, integrating the accelerator into a full embedded system is a natural progression. This would include connection to onboard sensors, storage, and interfaces for evaluating real-world performance. Building a reusable testbench with golden-output verification would also streamline development for new datasets, models, or platforms. Additional directions include exploring dynamic pruning or online reconfiguration for runtime adaptability, further improving efficiency under varying conditions.

5.1 Economic, Legal, Social, Ethical and Environmental Context

This project contributes to the growing demand for energy-efficient AI by implementing a low-power CNN accelerator on FPGA hardware. As AI inference is expected to dominate global compute energy consumption in the coming decade, the use of quantisation and hardware specialisation in this work supports sustainability goals by significantly reducing power usage compared to GPU alternatives.

Legally and ethically, potential applications such as license plate recognition must comply with privacy and data protection regulations. Although this work focuses solely on the inference engine, any future deployment must ensure adherence to relevant standards, such as the General Data Protection Regulation (GDPR), particularly in public or surveillance-facing use cases.

From an economic standpoint, the use of commercially accessible hardware (Zybo Z7) shows that high-performance inference is achievable without costly data centre infrastructure, enabling scalable AI solutions in resource-constrained environments and emerging markets.

The design adheres to established IP development practices, using AXI-compliant interfaces and modular HLS components, and was implemented entirely with open-source or properly licensed tools and pretrained models. This modular architecture not only enables rapid prototyping and reuse across projects but also allows for integration into broader training and deployment pipelines.

In particular, the focus on the inference phase—and the hardware-aware simplifications made—enable seamless integration into larger training regimes. This facilitates experimentation with transfer learning, hybrid quantisation strategies, and hardware-in-the-loop validation, thereby supporting higher accuracy and robustness in future models. As such, the work's impact extends beyond the specific implementation, providing a practical and ethical foundation for scalable, sustainable AI deployment in real-world edge systems.

Appendix A

Appendix: Additional Materials

A.1 Algorithms and Pseudocode

Algorithm 1 2D Convolution with Stride and Output-Side Padding

```
procedure CONV2D(input, kernel, bias, stride, out_H, out_W)
    H, W ← height and width of input
    K ← kernel size (assumed square)
    output ← zero matrix of size [out_H][out_W]
    for i = 0 to out_H − 1 do
        for j = 0 to out_W − 1 do
            in_row ← i · stride
            in_col ← j · stride
            if in_row + K ≤ H and in_col + K ≤ W then
                sum ← 0
                for m = 0 to K − 1 do
                    for n = 0 to K − 1 do
                        sum ← sum + input[in_row + m][in_col + n] · kernel[m][n]
                    end for
                end for
                output[i][j] ← sum + bias
            else
                output[i][j] ← 0                                  ▷ Zero-padded output
            end if
        end for
    end for
    return output
end procedure
```

Algorithm 2 Output First Multichannel Convolution

```
Initialize output feature maps O[o][x][y] ← 0 for all o, x, y
Initialise local input buffer
for each OC o in 0 → OC − 1 do
    Initialize accumulation buffer O[x][y] ← 0 for all x, y
    for each IC i in 0 → IC − 1 do
        Pass stream into line and window buffer
        Load kernel weights W_{o,i} for input i
        Compute partial convolution and accumulate
        if i = IC − 1 then
            Write to output stream O[x][y]
        end if
    end for
end for
```

Note: OC = Output Channels, IC = Input Channels

Algorithm 2 input buffer size of $IC \cdot H \cdot W$, where IC is the number of input channels, and H and W are the height and width of the input feature map. The temporary output accumulation buffer is of size $H \cdot W$. The algorithm processes one output channel at a time, reading all input channels for that output channel before writing the result to the output stream.

Algorithm 3 Some At Once Multichannel Convolution

```
Initialize output feature maps O[o][x][y] ← 0 for all o, x, y
Initialise local input buffer
for each OC block o in OC/UF do
    for each IC i in 0 → IC − 1 do
        Pass stream into line and window buffer
        for each uf in 0 → UF − 1 do
            Load kernel weights W_{o·UF+uf,i}
            Compute partial convolution for output o · UF + uf
            if i = IC − 1 then
                Write O[o · UF + uf][x][y] to stream
            end if
        end for
    end for
end for
```

Note: OC = Output Channels, IC = Input Channels, UF = Unroll Factor

Algorithm 3 is a hybrid approach that processes a subset of output channels in parallel, allowing for some degree of pipelining while still maintaining a manageable buffer size. The unroll factor (UF) determines how many output channels are processed simultaneously, balancing the trade-off between resource usage and performance.

Algorithm 4 Softmax / Argmax Output Selection

```
procedure OutputLayer(input, mode)
    C ← length(input)
    output ← array of size C
    if mode = "ARGMAX" then
        max_val ← −∞
        max_index ← −1
        for i = 0 to C − 1 do
            if input[i] > max_val then
                max_val ← input[i]
                max_index ← i
            end if
        end for
        return max_index                                  ▷ Class ID with max score
    else if mode = "SOFTMAX" then
        sum ← 0
        for i = 0 to C − 1 do
            output[i] ← exp(input[i])                     ▷ Use LUT or approximation in hardware
            sum ← sum + output[i]
        end for
        for i = 0 to C − 1 do
            output[i] ← output[i] / sum                   ▷ Fixed-point division
        end for
        return output                                     ▷ Probability distribution
    else
        error: Invalid mode
    end if
end procedure
```

Algorithm 5 Final Fully Connected Layer

```
procedure FullyConnected(input, weights, biases)
    N ← length(input)
    C ← number of output classes
    output ← array of size C
    for c = 0 to C − 1 do
        sum ← 0
        for i = 0 to N − 1 do
            sum ← sum + input[i] · weights[c][i]
        end for
        output[c] ← sum + biases[c]
    end for
    return output
end procedure
```

Algorithm 6 2D Pooling Function (Average/Max)

```
1:  procedure POOLING2D(input, kernel_size, stride, pooling_mode)
2:      H, W ← height and width of input
3:      k ← kernel_size
4:      s ← stride
5:      output ← empty tensor of size ⌊(H−k)/s + 1⌋ × ⌊(W−k)/s + 1⌋
6:      for i = 0 to H − k step s do
7:          for j = 0 to W − k step s do
8:              window ← input[i : i + k][j : j + k]
9:              if pooling_mode = "MAX" then
10:                 output[i/s][j/s] ← max(window)
11:             else if pooling_mode = "AVG" then
12:                 output[i/s][j/s] ← (1/(k×k)) Σ window
13:             else
14:                 error: Invalid pooling mode
15:             end if
16:         end for
17:     end for
18:     return output
19: end procedure
```

A.2 Figures

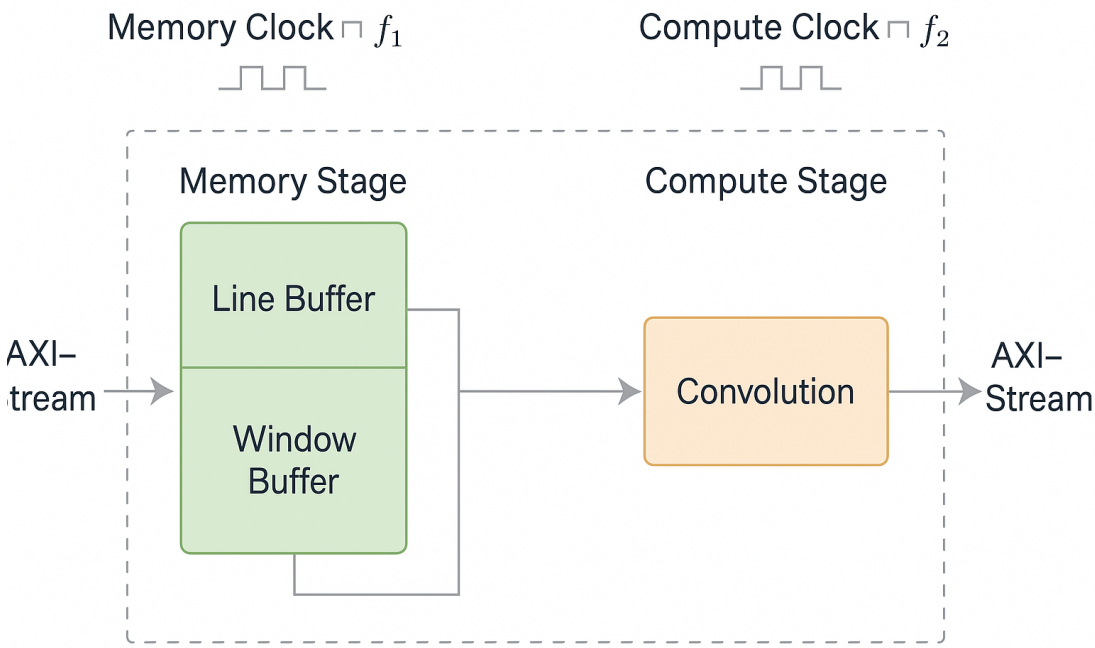

Figure A.1: Dual-clock architecture showing memory and compute stages operating under separate clocks. High-frequency input feeds line and window buffers, while convolution executes in a lower-frequency domain. CDC logic ensures safe data transfer.

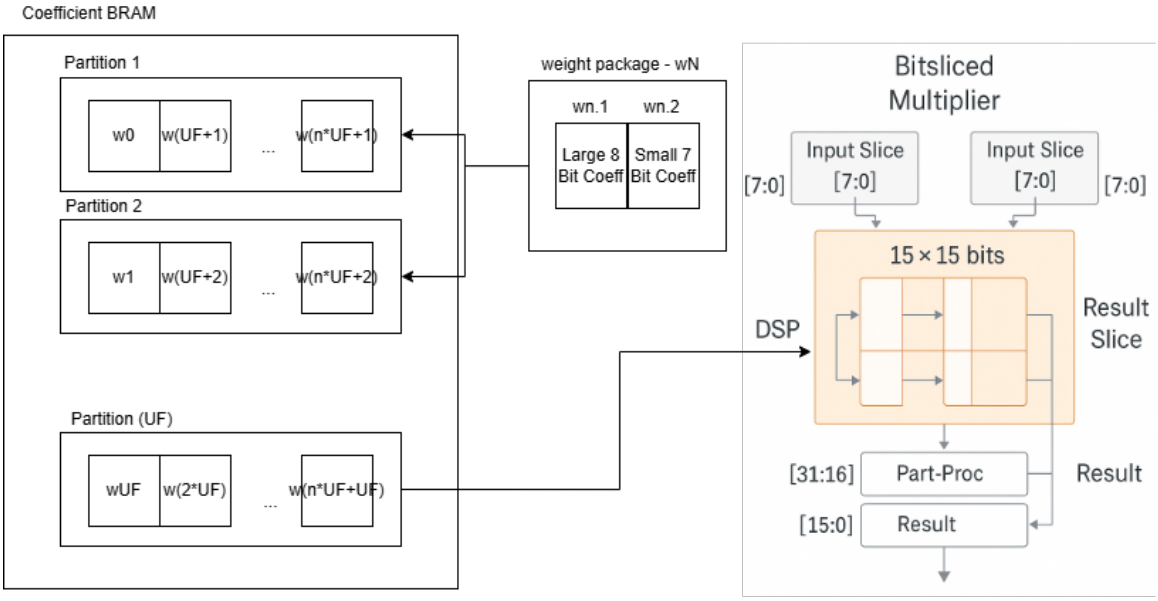

Figure A.2: Bitsliced multiplier and BRAM coefficient packaging. Coefficients are pre-packed into partitioned BRAMs aligned to the unrolling factor (UF), enabling parallel access for sliced DSP inputs. The DSP computes a fused result, which is then unpacked using post-processing logic (masking and shifting) to extract valid sub-results.

Bibliography

[1] "Ultra low latency and high performance deep learning processor with fpga," Alibaba Cloud Community, 2018. [Online]. Available: https://www.alibabacloud.com/blog/ultra-low-latency-and-high-performance-deep-learning-processor-with-fpga_593942

[2] R. F. Alvear-Sandoval, J. L. Sancho-Gómez, and A. R. Figueiras-Vidal, "On improving cnns performance: The case of mnist," *Information Fusion*, vol. 52, pp. 106–109, 12 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1566253518305293?casa_token=a5oa-4ISJckAAAAA:Xfcy49ZR5x8oTKcMS4S5ChbUi7LyiF7I4doaslX0GzZIVafyJSHJ2ikJ0tZS5LaWyA5wGfPFM98

[3] D. Bhatt, C. Patel, H. Talsania, J. Patel, R. Vaghela, S. Pandya, K. Modi, and H. Ghayvat, "Cnn variants for computer vision: History, architecture, application, challenges and future scope," *Electronics*, vol. 10, p. 2470, 10 2021.

[4] F. Chen, N. Chen, H. Mao, and H. Hu, "Assessing four neural networks on handwritten digit recognition dataset (mnist)," CHUANGXINBAN JOURNAL OF COMPUTING, 2018. [Online]. Available: https://arxiv.org/pdf/1811.08278

[5] A. Sateesan, S. Sinha, S. K. G, and A. P. Vinod, "A survey of algorithmic and hardware optimization techniques for vision convolutional neural networks on fpgas," *Neural Processing Letters*, vol. 53, pp. 2331–2377, 04 2021. [Online]. Available: https://link.springer.com/article/10.1007/s11063-021-10458-1

[6] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, pp. 541–551, 12 1989. [Online]. Available: https://direct.mit.edu/neco/article-abstract/1/4/541/5515/Backpropagation-Applied-to-Handwritten-Zip-Code?redirectedFrom=fulltext

[7] A. J. A. ElMaksoud, M. Ebbed, A. H. Khalil, and H. Mostafa, "Power efficient design of highperformance convolutional neural networks hardware accelerator on fpga: A case study with googlenet," *IEEE Access*, vol. 9, pp. 151 897–151 911, 2021.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *Thecvf.com*, pp. 770–778, 2016. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html

[9] F. He, T. Liu, and D. Tao, "Why resnet works? residuals generalize," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, pp. 5349–5362, 02 2020. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8984747?casa_token=dX95J2DPnUIAAAAA:mXEEGtfoftpKXO2jor6s67-rqEZk_hkmPFiZKlw57AJOc5KjkY2EZcaVP5vj3IgXc4Eu2Bs4oi8

[10] T. Moreau, T. Chen, and L. Ceze, "Leveraging the vta-tvm hardware-software stack for fpga acceleration of 8-bit resnet-18 inference," 06 2018. [Online]. Available: https://dl.acm.org/doi/10.1145/3229762.3229766

[11] Z. Li and S. Bilavarn, "Partial reconfiguration for energy-efficient inference on fpga: A case study with resnet-18," *2022 25th Euromicro Conference on Digital System Design (DSD)*, pp. 291–297, 08 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10741787

[12] M. Lead, Seng, B. Bang, C. Chiang, Z. Arabee, A. Salam, G. Tan, and H. Chai, "Mnist handwritten digit recognition with different cnn architectures," *Journal of Applied Technology and Innovation*, vol. 5, pp. 2600–7304, 2021. [Online]. Available: https://jati.sites.apiit.edu.my/files/2021/01/MNIST-Handwritten-Digit-Recognition-with-Different-CNN-Architectures.pdf#page=1.70

[13] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang, Y. Dai, J. Li, Z. Wang, R. Zhang, K. Wen, X. Ning, and Y. Wang, "Flightllm: Efficient large language model inference with a complete mapping flow on fpgas," arXiv.org, 2024. [Online]. Available: https://arxiv.org/abs/2401.03868

[14] P. Xiyuan, Y. Jinxiang, Y. Bowen, L. Liansheng, and P. Yu, "A review of fpga-based custom computing architecture for convolutional neural network inference," *Chinese Journal of Electronics*, vol. 30, pp. 1–17, 01 2021. [Online]. Available: https://ietresearch.onlinelibrary.wiley.com/doi/epdf/10.1049/cje.2020.11.002

[15] T. V. Huynh, "Deep neural network accelerator based on fpga," 11 2017. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8108073

[16] M. Vaithianathan, M. Patil, S. Ng, and S. Udkar, "Comparative study of fpga and gpu for high-performance computing and ai comparative study of fpga and gpu for high-performance computing and ai," *International Journal of Advancements in Computational Technology*, vol. 1, pp. 2583–8628, 2023.

[17] J. Si and S. L. Harris, "Handwritten digit recognition system on an fpga," *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*, 01 2018. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8301757?casa_token=ThOPlLIlOF4AAAAA:igp72QILaZ6p1lG4lwbnWXHEx1OrDb147AmsXp1WE4PgGJccoNxBgXP8mELhp74N4igvkxbYR5w

[18] Y. Zhao, J. Yi, and W. Guo, "Fpga-based cnn quantization method and hardware acceleration module," *2024 4th Asia-Pacific Conference on Communications Technology*

and Computer Science (ACCTCS), pp. 271–276, 02 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10612954

[19] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer cnn accelerators," 10 2016. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7783725

[20] J. Liang, T. Zhang, and G. Feng, "Channel compression: Rethinking information redundancy among channels in cnn architecture," *IEEE Access*, vol. 8, pp. 147 265–147 274, 01 2020. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9164969

[21] H.-J. Kang, "Aocstream: All-on-chip cnn accelerator with stream-based line-buffer architecture," arXiv.org, 2022. [Online]. Available: https://arxiv.org/abs/2212.11438

[22] S. Lahti and T. D. Hämäläinen, "High-level synthesis for fpgas - a hardware engineer's perspective," *IEEE Access*, pp. 1–1, 01 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10878985

[23] D. Giardino, M. Matta, F. Silvestri, S. Spanò, and V. Trobiani, "Fpga implementation of handwritten number recognition based on cnn," vol. 9, pp. 167–167, 02 2019.

[24] R. Xiao, J. Shi, and C. Zhang, "Fpga implementation of cnn for handwritten digit recognition," *2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)*, 05 2020. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9085002

[25] S. Zhu, H. Huang, Z. Hu, and Q. Tian, "Design of handwritten digit recognition system based on fpga," *International Conference on Signal Image Processing and Communication (ICSIPC 2021)*, p. 26, 06 2021. [Online]. Available: https://www.spiedigitallibrary.org/conference-proceedings-of-spie/11848/1184812/Design-of-handwritten-digit-recognition-system-based-on-FPGA/10.1117/12.2600178.full

[26] D. Parra, D. E. Sanabria, and C. Camargo, "A methodology and open-source tools to implement convolutional neural networks quantized with tensorflow lite on fpgas," *Electronics*, vol. 12, pp. 4367–4367, 10 2023. [Online]. Available: https://www.mdpi.com/2079-9292/12/20/4367

[27] G. Feng, Z. Hu, S. Chen, and F. Wu, "Energy-efficient and high-throughput fpga-based accelerator for convolutional neural networks," *2016 13th IEEE International Conference on Solid-State and Integrated Circuit Technology (ICSICT)*, pp. 624–626, 10 2016. [Online]. Available: https://ieeexplore.ieee.org/document/7998996

[28] M. R. Biswal, T. S. Delwar, A. Siddique, P. Behera, Y. Choi, and J.-Y. Ryu, "Pattern classification using quantized neural networks for fpga-based low-power iot devices," *Sensors*, vol. 22, pp. 8694–8694, 11 2022. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9699191/#:~:text=For%20network%20compression%2C%20QNN%20can,on%20an%20Xilinx%20ZCU102%20platform

[29] S. Diware, A. Singh, A. Gebregiorgis, R. V. Joshi, S. Hamdioui, and R. Bishnoi, "Accurate and energy-efficient bit-slicing for rram-based neural networks," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 7, pp. 164–177, 07 2022. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9840507